ParentPulse - AWS Architecture

https://dgm20st3o0yqp.cloudfront.net

School-parent communication platform | React SPA + Node.js/Express API on EC2, Amazon DynamoDB, served over HTTPS by Amazon CloudFront | Region ap-south-1 (Mumbai)

Users (web browsers)

Desktop or phone, any network

Parent
Dashboard, attendance, homework, announcements, reports, last known location of children

Teacher
Marks attendance, posts assignments + announcements

Student
Own attendance + assignments; shares device location (GPS) while the app is open

Admin
Users, classes, subjects, parent links, approvals

React 18 app runs in the browser; session kept in an HttpOnly cookie

maps, fonts

Third-party (loaded by browser)

OpenStreetMap tile servers (maps)
Google Fonts (web fonts)

1 HTTPS :443

Amazon CloudFront

Global edge network, PriceClass_100
Global CDN edge, TLS termination
Default *.cloudfront.net certificate
HTTP -> HTTPS redirect

/* cached static files
/api/* not cached; forwards cookies + all methods

Adds secret X-Origin-Verify header to every request sent to EC2
Cache invalidated on every deploy

2 HTTP :80 + secret header

cache invalidation + AWS API provisioning

AWS Cloud - Region ap-south-1 (Mumbai)

Default VPC (public subnet)

Security group parentpulse-app-sg

Inbound 80 : only from the CloudFront origin-facing prefix list
Inbound 22 : only from the Instance Connect Endpoint
Everything else closed (no public 443, no public SSH)

Amazon EC2 - t3.micro, Amazon Linux 2023

Reachable for web traffic only through CloudFront

nginx (port 80)
1. Rejects requests without the correct X-Origin-Verify header (403)
2. Serves the built React app (static files)
3. Adds HSTS, Content-Security-Policy, X-Frame-Options, etc.
4. Reverse-proxies /api/* to 127.0.0.1:4000

3 /api/* via localhost

Node.js 20 + Express API (port 4000, pm2)
helmet, CORS, body-size limit, per-IP rate limits
Auth: bcrypt, JWT session cookie (15 min idle / 8 h max), account lockout, session revocation
Role checks + data isolation on every route
Routes: auth, users, students, classes, subjects, assignments, attendance, announcements, notifications, reports, location
AWS SDK v3 (DynamoDB Document Client)

4 AWS SDK HTTPS

SSH :22

EC2 Instance Connect Endpoint

SSH into the VPC through the AWS API - no SSH port open to the internet. Needs AWS credentials + the EC2 key pair. Its own SG allows only port 22 to the app.

5 role credentials

AWS IAM

Role parentpulse-ec2-role
+ instance profile on EC2
Policy: DynamoDB read/write only on tables parentpulse_*
Temporary credentials from instance metadata - no keys stored on the server

Amazon DynamoDB

On-demand, encrypted at rest
Tables: parentpulse_<name>_prod

users
id | GSI email-index

students
id | GSI class, user

parent_student
parentId + studentId

classes
id

subjects
id | GSI class-index

assignments
id | GSI class, teacher

attendance
studentId + date

announcements
id

notifications
userId + sortKey

locations
studentId (latest only)

Developer laptop

Operations / deployment
deploy/aws/*.sh (bash + AWS CLI)
- builds the React app (Vite)
- ships code via SSH tunnel
- creates tables, restarts pm2
- invalidates CloudFront cache
GitHub repo: source code

6 SSH over the AWS API tunnel (aws ec2-instance-connect open-tunnel)
deploy: upload, npm install, pm2 restart

Legend: User / data traffic | CloudFront -> origin | IAM credentials | Operations / deployment | Third-party, fetched by the browser

Numbers 1-6 match the request flows described on page 2.

How the site works

Every request from a browser goes through CloudFront over HTTPS. The EC2 server accepts traffic only from CloudFront, and the API reaches DynamoDB with temporary IAM role credentials. Numbered steps refer to the arrows on page 1.

A. Opening the website

1. The browser requests **https://dgm20st3o0yqp.cloudfront.net**. CloudFront terminates TLS (HTTP is redirected to HTTPS). **[1]**
2. Static files (JavaScript, CSS) are served from the CloudFront edge cache when possible. Otherwise CloudFront fetches them from EC2 on port 80, adding the secret **X-Origin-Verify** header. **[2]**
3. The security group only admits CloudFront's IP ranges. nginx checks the secret header (returns 403 without it), serves the React build and adds the security headers (HSTS, CSP, X-Frame-Options, Permissions-Policy).
4. The React single-page app starts in the browser and calls **/api/auth/me** to see whether a session cookie exists.

B. Signing in and the session

1. **POST /api/auth/login** goes to CloudFront (the **/api/*** behaviour is never cached and forwards cookies), then nginx, then Express on port 4000. **[1] [2] [3]**
2. Express applies the per-IP rate limit, looks the user up in DynamoDB through the **email-index** GSI **[4]**, and compares the password with bcrypt. Five failures lock the account for 15 minutes.
3. On success it sets **pp_session**: a signed JWT in an HttpOnly, Secure, SameSite=Strict cookie that page scripts can never read.
4. The session expires after **15 minutes of inactivity** (the app warns one minute before) and **8 hours** after sign-in at most. Logging out or changing the password revokes every session for that user.

C. Every data request (dashboards, attendance, assignments...)

1. The browser sends the cookie automatically. The **requireAuth** middleware verifies the JWT and the 8-hour limit, then reads the user row (active status and token version), so deactivation takes effect immediately.
2. Role checks (RBAC) and data-isolation checks (**authz.js**) make sure parents only see their own children, teachers only their classes, and so on.
3. The API queries DynamoDB with the AWS SDK **[4]**, using temporary credentials from the EC2 IAM role **[5]**. It returns JSON with **Cache-Control: no-store** and re-issues the cookie (sliding session).

D. Child location (last known position)

1. A student turns on location sharing once. The browser asks for permission (allowed only on HTTPS, Permissions-Policy geolocation=self).
2. While ParentPulse is open on the student's device it sends **PUT /api/location/me** (at most every 10 s, at least every 60 s, or after a 25 m move). Only the latest position is kept in **parentpulse_locations_prod**.
3. A parent opens **Location**. **GET /api/location/children** returns only children linked in **parent_student**, each with their last known position and its time ("Live" if updated in the last 3 minutes).
4. The map is drawn with Leaflet. Map tiles come straight from OpenStreetMap to the browser.

E. Deploying an update

1. On the developer laptop, **05-deploy-app.sh** builds the React app and packages the backend.
2. The files go up as one tar stream over SSH, tunnelled through the **EC2 Instance Connect Endpoint** (no public SSH port), into a staging folder. **[6]**
3. Only after the upload is complete does the server swap the files in, run npm install, create any missing DynamoDB tables, reload nginx and restart the API with pm2.
4. Finally the script invalidates the CloudFront cache so users get the new version straight away.

Services used

Service	Role in ParentPulse	Configuration
Amazon CloudFront	Public HTTPS entry point and CDN	Default certificate, HTTP->HTTPS redirect, /api/* uncached with cookies forwarded, secret origin header, PriceClass_100
Amazon EC2	Application server	t3.micro, Amazon Linux 2023, nginx + Node.js 20 + pm2, ap-south-1
Amazon DynamoDB	Database	10 tables, on-demand capacity, global secondary indexes for lookups
AWS IAM	Server permissions	EC2 role + instance profile, access limited to parentpulse_* tables
Amazon VPC + security groups	Network isolation	Default VPC. App SG: port 80 from the CloudFront prefix list only, port 22 from the Instance Connect Endpoint only
EC2 Instance Connect Endpoint	Administrator SSH access	SSH through the AWS API; no SSH port open to the internet
Managed prefix list	Origin lock-down	com.amazonaws.global.cloudfront.origin-facing
OpenStreetMap, Google Fonts	Map tiles and web fonts (non-AWS)	Loaded directly by the browser, allowed by the CSP

Security layers

- **Transport:** HTTPS only, HSTS, origin reachable only through CloudFront (IP allow-list + secret header).
- **Identity:** bcrypt-hashed passwords with a strength policy; rate limits, account lockout, admin approval for teachers.
- **Sessions:** HttpOnly + Secure + SameSite=Strict cookie, 15 min idle / 8 h max, server-side revocation.
- **Authorisation:** role checks and per-record ownership checks on every API route.
- **Browser hardening:** Content-Security-Policy, X-Frame-Options DENY, nosniff, Referrer- and Permissions-Policy.
- **Infrastructure:** no stored AWS keys (IAM role), least-privilege table access, no public SSH.

Application stack

Layer	Technology
Frontend	React 18, Vite, React Router 6, Leaflet (maps), custom CSS design system with light/dark themes
Backend	Node.js 20, Express 4, helmet, express-rate-limit, bcryptjs, jsonwebtoken, AWS SDK v3 (DynamoDB)
Web server	nginx (static files, reverse proxy, security headers), pm2 process manager with systemd start-up
Tooling	Bash + AWS CLI v2 deploy scripts (deploy/aws/01-08), GitHub for source control